



Universidade de Brasília / Faculdade do Gama
Engenharia de Software
Disciplina FGA0314 – Testes de Software

Relatório de Auditoria de Testes – Módulo 3

Atividades 1, 2 e 3 – Auditoria dos Testes de Unidade do Projeto NoFluxo UnB

Integrante	Matrícula
Enzo Lopes Ferreira	232002000
Vinícius de Jesus Bessa Fernandes	222006490
Vitor Marconi Trancoso Albuquerque	222006202

Brasília, maio de 2026

Contexto da auditoria

Esta equipe atua como **colaboradora externa** auditando os testes do projeto **NoFluxo UnB**. A equipe *não* desenvolveu o código de produção: o trabalho aqui descrito é de *análise* dos testes existentes (Atividades 1 e 2) e de *proposta* de uma reescrita exemplar (Atividade 3).

O projeto auditado é composto por:

- Um **backend Node.js/TypeScript** (`no_fluxo_backend`) que expõe controllers Express e usa Supabase como banco;
- Um **serviço Python** de IA (`ai_agent`) baseado em Flask, integrado ao RAGFlow;
- Scripts **Python** de scraping/coleta de dados do SIGAA;
- Frontends em **Flutter** e **Svelte**.

A auditoria concentrou-se em duas áreas com testes *de unidade*: `no_fluxo_backend/tests-ts/` (Jest + TypeScript) e `tests-python/` (Pytest + unittest). Frontends e scripts sem testes de unidade foram excluídos do escopo.

1 Atividade 1 – Anatomia dos Testes do Projeto

O objetivo desta atividade é mapear, do ponto de vista da auditoria, como o projeto NoFluxo organiza seus testes de unidade. A equipe analisou as quatro dimensões pedidas no enunciado: convenção de nomes, estrutura AAA, uso de mocks/stubs e dependências externas.

1.1 Estrutura geral dos testes

Área	Framework	Localização
Backend TS (Express)	Jest 29.7.0 + ts-jest	no_fluxo_backend/tests-ts/*.test.ts
Serviço Python (Flask)	Pytest + pytest-mock	tests-python/test_*.py
Scrapers Python	unittest + mock	tests-python/test_extrair_*.py

1.2 Convenção de nomes

No **backend TypeScript** o projeto segue a convenção do Jest com `describe` agrupando por controller e `it` descrevendo o comportamento em inglês na forma “*should + verbo + condição*”:

```
1 // 'describe' agrupa os testes de um mesmo controller (boa pratica)
2 describe('CursosController', () => {
3   // Cada 'it' descreve UM comportamento -- formato "should + acao +
4     condicao"
5   it('should return 200 with cursos data when successful', async () => {
6     ... });
7   it('should return 500 when cursos query fails', async () => { ... });
8   it('should return 500 when credits_por_curso query fails', async ()
9     => { ... });
10 });
11 // >>> Observacao da auditoria: nomes em ingles, foco em comportamento
12 -- OK
```

Listing 1: cursos_controller.test.ts – linhas 22, 42, 74, 90

Nos **testes Python** a convenção é o prefixo `test_` seguido por uma descrição em **português** usando `snake_case`:

```
1 # Convencao Python: prefixo test_ + descricao em portugues, snake_case
2 # 'client' e 'mock' sao fixtures injetadas pelo pytest
3 def test_analisar_materia_endpoint_sucesso(client, mocker):
4     """Testa o endpoint /assistente em um cen rio de sucesso..."""
5 # >>> Observacao da auditoria: o nome cita o endpoint e o cenario,
6 #     mas nao explicita o RESULTADO esperado (slide 24 da aula)
```

Listing 2: tests-python/test_app.py – linha 55

Avaliação da auditoria: a convenção do backend TS está alinhada às boas práticas (foco em comportamento). Já a Python está parcialmente alinhada – os nomes descrevem comportamento, mas em alguns testes herdados de `unittest` usam-se nomes muito genéricos como `test_remove_acentos` ou `test_coleta_dados`, que focam em *nome do método* e não no comportamento esperado, exatamente o problema descrito no slide 23 da aula.

1.3 Estrutura AAA (Arrange / Act / Assert)

A equipe identificou que os testes do backend TS seguem AAA de forma **implícita**: o padrão está presente, mas *sem* comentários demarcando os blocos. Exemplo extraído do projeto:

```

1 it('should return 200 with cursos data when successful', async () => {
2     // ===== ARRANGE =====
3     // (no projeto auditado NAO ha esses comentarios -- estamos
4     // destacando os blocos AAA para fins didaticos)
5     const mockCursosData = [ { id_curso: 1, nome_curso: 'Administracao',
6     }, ... ];
7
8     // Configura DUAS respostas sequenciais do banco mockado:
9     // 1a chamada -> dados de cursos
10    // 2a chamada -> dados de creditos por curso
11    (SupabaseWrapper.get().from('cursos').select as jest.Mock)
12    .mockResolvedValueOnce({ data: mockCursosData, error: null });
13    .mockResolvedValueOnce({ data: mockCreditosData, error: null });
14
15    // ===== ACT =====
16    // UMA unica chamada ao handler do controller -- boa pratica
17    const handler = CursosController.routes["all-cursos"].value;
18    await handler(mockRequest as Request, mockResponse as Response);
19
20    // ===== ASSERT =====
21    // Verifica status HTTP + corpo da resposta
22    expect(statusSpy).toHaveBeenCalledWith(200);
23    expect(jsonSpy).toHaveBeenCalledWith([ ... ]);
24 });

```

Listing 3: cursos_controller.test.ts – linhas 42–72 (AAA implícito)

Nos testes Python a equipe observou que o padrão AAA também aparece, mas é frequentemente **misturado**: o setup do cliente Flask é uma *fixture*, mocks podem aparecer espalhados, e em alguns casos há vários blocos de Act + Assert no mesmo teste.

1.4 Uso de mocks/stubs

Backend TS: o projeto usa exclusivamente *Jest mocks* (`jest.fn()`, `jest.mock()`, `jest.spyOn()`). A estratégia adotada é mockar o módulo `SupabaseWrapper` no topo de cada arquivo de teste, evitando qualquer chamada real ao banco:

```

1 // Substitui o modulo 'supabase_wrapper' inteiro por um dubl
2 // (NENHUMA chamada real ao banco acontece nos testes)
3 jest.mock('../src/supabase_wrapper', () => {
4     // Cada nivel da cadeia .from(...).select(...) precisa ser mockado
5     const mockSelect = jest.fn();
6     const mockFrom = jest.fn(() => ({ select: mockSelect }));
7     const mockGet = jest.fn(() => ({ from: mockFrom }));
8
9     // Retorna o objeto que substitui o modulo original
10    return { SupabaseWrapper: { get: mockGet } };
11 });
12 // >>> Auditoria: bom isolamento, MAS o teste fica acoplado a forma
13 // exata como o controller chama o Supabase (quebra em refactor)

```

Listing 4: cursos_controller.test.ts – linhas 6–20 (mock do Supabase)

Python: o projeto usa `unittest.mock` (`@patch`, `MagicMock`, `mock_open`) e o auxiliar `pytest-mock` (`mock.patch`). Exemplo encontrado em `test_app.py`:

```

1 # 'mock.patch' substitui RagflowClient (chamada HTTP externa)
2 # por um dubl controlado pelo teste
3 mock_ragflow_client = mocker.patch(
4     'no_fluxo_backend.ai_agent.app.RagflowClient'
5 ).return_value
6
7 # Define respostas estaticas para os metodos usados no fluxo
8 mock_ragflow_client.start_session.return_value = "sessao_mock_123"
9 mock_ragflow_client.analyze_materia.return_value = {
10     "code": 0, "data": "dados_mock"
11 }
12 # >>> Auditoria: dubl bem aplicado -- isola API externa do RAGFlow

```

Listing 5: tests-python/test_app.py – linhas 60–63

1.5 Dependências externas

A tabela abaixo resume as dependências externas *detectadas pela auditoria* e como elas são tratadas nos testes existentes do projeto:

Dependência	Onde aparece	Tratamento nos testes
Banco Supabase	Backend TS, todos os controllers	Mockado via <code>jest.mock</code>
Request/Response Express	Backend TS	Mock manual via <code>Partial<Request></code>
RAGFlow API (HTTP)	<code>ai_agent/app.py</code>	<code>mocker.patch</code>
SIGAA (HTTP scraping)	<code>coleta_dados/scraping/</code>	<code>@patch('requests.Session')</code>
Sistema de arquivos	Salvar/ler JSON e CSV	<code>mock_open</code> , <code>patch('builtins.open')</code>
Variáveis de ambiente	<code>config.py</code>	<code>monkeypatch</code> (pytest)
Relógio/data atual	Não há testes que dependam disso	N/A

Conclusão da auditoria (Atividade 1): o projeto *já adota parcialmente* as boas práticas vistas em aula – nomeação por comportamento, AAA, e mocks para isolar dependências externas. O backend TS é a área mais alinhada com as recomendações; a parte Python apresenta mais inconsistências (nomes genéricos, AAA misturado) e alguns testes com setups muito longos, o que será explorado nas Atividades 2 e 3.

2 Atividade 2 – Encontrar um “Teste Ruim”

2.1 Teste escolhido pela auditoria

A equipe selecionou o teste “should return 200 with database test results when successful” do bloco `describe('banco')`, localizado no arquivo `no_fluxo_backend/tests-ts/teste` (linhas 67–130). Trata-se de um teste já presente no projeto que acumula vários sintomas de “teste ruim” descritos nos slides 22–25 da aula.

2.2 Código original (extraído do projeto auditado)

```

1 describe('banco', () => {
2     // [PROBLEMA 6] Nome generico: nao diz QUAL cenario gera o 200

```

```

3      it('should return 200 with database test results when successful',
      async () => {
4
5          // [PROBLEMA 5] Dados de teste excessivos -- nada disso eh
6          // verificado nos asserts; aumenta carga cognitiva
7          const mockCursosData = [
8              { nome_curso: 'Administracao' },
9              { nome_curso: 'Engenharia' }
10         ];
11
12         const mockMateriasData = [
13             {
14                 nome_curso: 'Administracao',
15                 materias_por_curso: [
16                     {
17                         id_materia: 1,
18                         nivel: 1,
19                         materias: {
20                             id_materia: 1,
21                             nome_materia: 'Matematica',
22                             codigo_materia: 'MAT001'
23                         }
24                     },
25                     {
26                         id_materia: 2,
27                         nivel: 2,
28                         materias: {
29                             id_materia: 2,
30                             nome_materia: 'Portugues',
31                             codigo_materia: 'PORT001'
32                         }
33                     }
34                 ]
35             }
36         ];
37
38         // [PROBLEMA 4] Os mocks abaixo modelam a forma EXATA como o
39         // controller chama o Supabase (.from -> .select -> .limit).
40         // Qualquer refactor interno quebra o teste.
41         const mockSelectChain1 = jest.fn().mockResolvedValueOnce({
42             data: mockCursosData,
43             error: null,
44         });
45
46         const mockSelectChain2 = jest.fn().mockResolvedValueOnce({
47             data: mockMateriasData,
48             error: null,
49         });
50
51         const mockLimitChain = jest.fn().mockReturnValue({
52             select: mockSelectChain2,
53         });
54
55         // [PROBLEMA 3] Arrange gigante e sem separacao; AAA impl cito
56         mockFrom.mockReturnValueOnce({
57             select: mockSelectChain1,
58         }).mockReturnValueOnce({
59             select: jest.fn().mockReturnValue({

```

```

60         limit: mockLimitChain,
61     }},
62 });
63
64     // ----- aqui acaba o Arrange e começa Act + Assert (sem
demarcacao) -----
65     const handler = TestesController.routes["banco"].value;
66     await handler(mockRequest as Request, mockResponse as Response);
67
68     // [PROBLEMA 2] TRES asserts verificando coisas distintas:
69     // status HTTP, identificador do teste, status logico.
70     // Quando falha, nao se sabe QUAL comportamento foi violado.
71     expect(statusSpy).toHaveBeenCalled(200);
72     const responseData = jsonSpy.mock.calls[0][0];
73     expect(responseData.teste).toBe('conexao_banco');
74     expect(responseData.status).toBe('sucesso');
75     // [PROBLEMA 7] Nao ha cobertura de valores limite (lista vazia,
76     // erro de query, 1 unico curso) -- so o caminho feliz
77 });
78 });
79 // [PROBLEMA 1] O teste inteiro tem ~65 linhas (>90% sao setup de mock)
80 // [PROBLEMA 8] mockMateriasData se repete em describe('curso') e
81 // describe('casamento') -- duplicacao indica falta de test data builder

```

Listing 6: testes_controller.test.ts – linhas 67–130 (teste original “ruim”) – comentarios da auditoria em destaque

2.3 Problemas identificados pela auditoria

1. **Difícil de entender (mais de 60 linhas).** O teste tem ~65 linhas. A maior parte é setup de mock; o comportamento testado real (uma chamada e três asserts) está enterrado ao final.
2. **Múltiplos asserts verificando coisas diferentes.** `expect(statusSpy).toHaveBeenCalled()`, `expect(responseData.teste).toBe(...)` e `expect(responseData.status).toBe(...)` testam *três* aspectos distintos. Quando o teste falha, não fica claro *qual* comportamento foi violado – o nome do teste só promete “return 200”.
3. **Mistura Arrange/Act/Assert.** Não há separação clara entre os três blocos. O Arrange é gigante, sem comentários, e a inicialização `mockFrom.mockReturnValueOnce(...)` é encadeada com lógica de “primeira chamada/segunda chamada” que esconde a regra de negócio testada.
4. **Depende fortemente da implementação interna.** Os mocks modelam a forma exata como o controller chama o Supabase (`from -> select -> limit -> select`). Qualquer refactor interno que mantenha o mesmo comportamento externo *quebra* o teste – problema clássico discutido no slide 23 da aula (“foca na implementação”).
5. **Dados de teste excessivos e irrelevantes.** O teste cria duas matérias completas (Matematica, Portugues) com níveis e códigos – nada disso é verificado nos asserts. Isso aumenta a carga cognitiva sem agregar valor.
6. **Nome genérico, não comunica o cenário.** “should return 200 with database test results when successful” diz *o que* retorna, mas não *em que cenário* (cursos

encontrados? base vazia? quantos itens?). Pelo padrão da aula (slide 24), o nome deveria descrever o comportamento de forma inequívoca.

7. **Não cobre valores limite.** Não há testes para: lista de cursos vazia, erro na segunda query (`materias_por_curso`), ou um único curso retornado. Apenas o caminho “feliz” está coberto.
8. **Setup duplicado entre testes do mesmo describe.** O mesmo `mockMateriasData` aparece praticamente igual em `describe('curso')` e `describe('casamento')`, indicando falta de *test data builders*.

3 Atividade 3 – Reescrita do Teste

A equipe propõe, a seguir, uma reescrita do teste auditado. A proposta aplica AAA explícito, melhora a nomenclatura, separa o teste em casos focados (*um act, um comportamento*) e adiciona casos de valores limite (lista vazia, erro de banco), conforme pedido no enunciado. Esta seção representa a contribuição direta da equipe auditora ao projeto – e é a única do relatório escrita em primeira pessoa.

3.1 Test data builder e helper de mock

Primeiro, extraímos os dados e o setup repetido para um helper. Isso reduz o ruído nos testes propostos e remove duplicação que a auditoria identificou no arquivo original.

```
1 // =====
2 // HELPERS DE TESTE -- declarados no topo do arquivo
3 // Objetivo: tirar o ruído de mock dos casos de teste e remover
4 // duplicacao identificada pela auditoria (PROBLEMA 8).
5 // =====
6
7 // [BUILDER] Cria um curso minimo, so com o que importa pro teste
8 const aCurso = (nome: string) => ({ nome_curso: nome });
9
10 // [HELPER] Encapsula o mock da 1a query (lista de cursos)
11 //   data -> resposta de sucesso
12 //   error -> caso de erro de banco (default: null = sucesso)
13 const stubCursosQuery = (data: any, error: any = null) => {
14   const select = jest.fn().mockResolvedValueOnce({ data, error });
15   mockFrom.mockReturnValueOnce({ select });
16 };
17
18 // [HELPER] Encapsula o mock da 2a query (materias_por_curso)
19 // Esconde o encadeamento .from -> .select -> .limit -> .select
20 // dos casos de teste -- assim, mudancas internas afetam apenas
21 // este helper, nao todos os testes.
22 const stubMateriasPorCursoQuery = (data: any, error: any = null) => {
23   const select = jest.fn().mockResolvedValueOnce({ data, error });
24   const limit = jest.fn().mockReturnValue({ select });
25   mockFrom.mockReturnValueOnce({
26     select: jest.fn().mockReturnValue({ limit }),
27   });
28 };
```

Listing 7: Helpers propostos pela equipe – reduzem setup repetido

3.2 Testes reescritos pela equipe auditora

Na proposta da equipe, cada teste foca em *um* comportamento, segue AAA explícito e cobre um cenário limite distinto.

```
1 // =====
2 // SUITE REESCRITA PELA EQUIPE AUDITORA
3 // Os 5 testes abaixo SUBSTITUEM o teste unico original (~65 linhas)
4 // e cobrem caminho feliz + 4 cenarios de borda.
5 // =====
6 describe('TestesController.banco (teste de conexao com o banco)', () =>
7 {
8     // -----
9     // CASO 1 -- caminho feliz com varios cursos
10    // -----
11    it('Banco_com_cursos_e_materias_retorna_status_sucesso', async () =>
12    {
13        // Arrange -- prepara 2 cursos e 1 entrada de materias
14        stubCursosQuery([aCurso('Administracao'), aCurso('Engenharia')])
15        ;
16        stubMateriasPorCursoQuery([
17            nome_curso: 'Administracao',
18            materias_por_curso: [],
19        ]]);
20
21        // Act -- UMA unica chamada ao handler (slide 13)
22        await TestesController.routes["banco"].value(
23            mockRequest as Request,
24            mockResponse as Response,
25        );
26
27        // Assert -- objectContaining: nao quebra se a resposta ganhar
28        // novos campos no futuro (asserts focados em comportamento)
29        expect(statusSpy).toHaveBeenCalled(200);
30        expect(jsonSpy).toHaveBeenCalled(
31            expect.objectContaining({
32                teste: 'conexao_banco',
33                status: 'sucesso',
34            })
35        );
36    });
37
38    // -----
39    // CASO 2 -- valor limite: ZERO cursos cadastrados
40    // -----
41    it('
42    Banco_sem_cursos_cadastrados_retorna_status_sucesso_com_lista_vazia',
43    async () => {
44        // Arrange -- limite inferior: lista vazia
45        stubCursosQuery([]);
46        stubMateriasPorCursoQuery([]);
47
48        // Act
49        await TestesController.routes["banco"].value(
50            mockRequest as Request,
51            mockResponse as Response,
52        );
53    });
54 }
```



```

49
50 // Assert -- mesmo sem cursos, conexao deve responder sucesso
51 expect(statusSpy).toHaveBeenCalled(200);
52 expect(jsonSpy).toHaveBeenCalled(
53     expect.objectContaining({
54         teste: 'conexao_banco',
55         status: 'sucesso',
56     }),
57 );
58 });
59
60 // -----
61 // CASO 3 -- valor limite: EXATAMENTE 1 curso
62 // -----
63 it('Banco_com_um_unico_curso_retorna_status_sucesso', async () => {
64     // Arrange -- limite minimo de uma colecao nao vazia
65     stubCursosQuery([aCurso('Engenharia')]);
66     stubMateriasPorCursoQuery([
67         {
68             nome_curso: 'Engenharia',
69             materias_por_curso: [],
70         }
71     ]);
72
73     // Act
74     await TestesController.routes["banco"].value(
75         mockRequest as Request,
76         mockResponse as Response,
77     );
78
79     // Assert -- foco em: handler nao falha com colecao de 1 item
80     expect(statusSpy).toHaveBeenCalled(200);
81 });
82
83 // -----
84 // CASO 4 -- caminho de ERRO: 1a query (cursos) falha
85 // -----
86 it('Banco_quando_query_de_cursos_falha_retorna_500_com_mensagem_de_erro',
87     async () => {
88         // Arrange -- simula erro retornado pelo Supabase
89         const erro = { message: 'Database error' };
90         stubCursosQuery(null, erro);
91
92         // Act
93         await TestesController.routes["banco"].value(
94             mockRequest as Request,
95             mockResponse as Response,
96         );
97
98         // Assert -- payload de erro deve conter mensagem original
99         expect(statusSpy).toHaveBeenCalled(500);
100         expect(jsonSpy).toHaveBeenCalled({
101             teste: 'conexao_banco',
102             status: 'erro',
103             erro: erro.message,
104         });
105     });
106
107 // -----

```

```

105 // CASO 5 -- caminho de ERRO: 2a query (materias) falha
106 // (este caso NAO existia no teste original)
107 // -----
108 it('
Banco_quando_query_de_materias_falha_retorna_500_com_mensagem_de_erro
', async () => {
109     // Arrange -- 1a query OK, 2a query erro
110     stubCursosQuery([aCurso('Administracao')]);
111     stubMateriasPorCursoQuery(null, { message: 'materias error' });
112
113     // Act
114     await TestesController.routes["banco"].value(
115         mockRequest as Request,
116         mockResponse as Response,
117     );
118
119     // Assert -- basta verificar status 500 + flag de erro logica
120     expect(statusSpy).toHaveBeenCalled(500);
121     expect(jsonSpy).toHaveBeenCalledWith(
122         expect.objectContaining({ status: 'erro' }),
123     );
124 });
125 });

```

Listing 8: Versão reescrita aplicando AAA, melhor nomenclatura e cobertura de valores limite

3.3 Melhorias propostas pela equipe auditora

1. **AAA explícito.** Cada teste proposto contém comentários `// Arrange`, `// Act` e `// Assert` demarcando os blocos – aumenta a legibilidade (slide 18 da aula).
2. **Nomes que descrevem comportamento.** Em vez de `should return 200 with database test results when successful`, a equipe propõe `Banco_com_cursos_e_materias_retorna_500_com_mensagem_de_erro`. O nome descreve *o cenário* (com cursos e matérias) e *o resultado esperado* (status sucesso), seguindo a recomendação do slide 24 (“Use frases simples e expressivas”) e do slide 25 (separar palavras com *underscore*).
3. **Cobertura de valores limite.** O teste original do projeto cobria apenas o caminho feliz. A versão proposta pela equipe adiciona:
 - Lista de cursos *vazia* (zero elementos);
 - *Um único* curso (limite mínimo de uma coleção não vazia);
 - Falha na primeira query;
 - Falha na segunda query (`materias_por_curso`).
4. **Um Act por teste.** Cada teste proposto tem exatamente uma chamada ao handler – prática recomendada no slide 13 (“Um bom teste normalmente possui apenas UM Act”).
5. **Asserts focados em comportamento, não em estrutura.** `expect.objectContaining({ status: 'sucesso' })` verifica apenas o que importa para aquele teste, em vez de

exigir o JSON completo. Isso reduz a fragilidade quando campos auxiliares são adicionados/removidos da resposta.

6. **Helpers reduzem ruído de setup.** A equipe introduziu `stubCursosQuery` e `stubMateriasPorCurso` para encapsular a forma como os mocks são encadeados, permitindo reuso entre testes e mantendo o foco do caso de teste no *cenário*, não no detalhe de implementação.
7. **Independência total.** Cada teste proposto prepara seus próprios mocks (graças ao `afterEach` com `jest.clearAllMocks()`) e não depende da ordem de execução – atende ao requisito do slide 19 (“um teste deve poder executar sozinho, em qualquer ordem”).

Conclusão da auditoria

A auditoria realizada pela equipe colaboradora mostrou que o projeto NoFluxo já adota *parcialmente* as boas práticas vistas no Módulo 3 – isolamento via mocks, AAA implícito e nomes razoáveis no backend TypeScript. Os pontos fracos detectados foram: (i) testes longos com setup excessivo; (ii) acoplamento dos mocks à implementação interna do Supabase (testes que dependem do encadeamento exato `from -> select -> limit`); (iii) ausência de testes para valores limite e caminhos de erro nos testes do `TestesController`; e (iv) nomes que descrevem o método/retorno em vez do comportamento.

A reescrita proposta na Atividade 3 ilustra como aplicar diretamente as recomendações da aula (AAA explícito, nomes expressivos, um Act por teste, cobertura de limites) *sem* alterar o código de produção do NoFluxo – ganhando legibilidade e cobertura ao mesmo tempo. A equipe recomenda que essa abordagem seja estendida para os demais arquivos de teste do projeto em ciclos futuros de auditoria.